

ALGORITMOS DE HASHING: ANÁLISE COMPARATIVA

HASHING ALGORITHMS: A COMPARATIVE ANALYSIS

Prof. Ms. José Augusto Navarro Garcia Manzano - ORCID: 0000-0001-9248-7765

augusto.garcia@ifsp.edu.br

Instituto Federal de Educação, Ciência e Tecnologia de São Paulo (IFSP)

<https://zenodo.org/records/20454279> - DOI: 10.5281/zenodo.20454279

Resumo

As funções hash são mecanismos fundamentais da computação contemporânea, amplamente empregadas em estruturas de dados, indexação e sistemas de segurança. Este artigo apresenta uma análise comparativa entre duas abordagens distintas de funções de dispersão para cadeias de caracteres: uma baseada em ponderação linear (somas sucessivas) e outra baseada em multiplicação sucessiva inspirada no algoritmo clássico DJB2. A metodologia combina a modelagem matemática estrutural dos algoritmos com uma validação empírica realizada em linguagem C++, testando exaustivamente todas as combinações possíveis de quatro caracteres maiúsculas mapeadas em tabelas de dispersão em memória (`unordered_map`). Os resultados demonstram que, enquanto o modelo linear exibe crescimento estrito, baixa sensibilidade à ordem dos caracteres (vulnerabilidade a anagramas) e elevada taxa de colisões, o algoritmo DJB2 distribui os dados de forma significativamente mais uniforme e mitiga drasticamente a ocorrência de colisões devido ao seu comportamento geométrico e multiplicativo. Conclui-se que pequenas modificações na modelagem aritmética impactam diretamente a ocupação do espaço de bits e a eficiência computacional, evidenciando o rigor lógico necessário no ensino e projeto de estruturas de dados.

Palavras-chave: algoritmos de dispersão, análise comparativa, função hash, taxa de colisão.

Abstract

Hash functions are fundamental mechanisms in contemporary computing, widely employed in data structures, indexing, and security systems. This paper presents a comparative analysis between two distinct approaches to string hashing functions: one based on linear weighting (successive additions) and another based on successive multiplication inspired by the classic DJB2 algorithm. The methodology combines the structural mathematical modeling of the algorithms with an empirical validation conducted in the C++ programming language, exhaustively testing all possible four-letter uppercase combinations mapped into in-memory hash tables (`unordered_map`). The results demonstrate that while the linear model exhibits strict linear growth, low sensitivity to character ordering (vulnerability to anagrams), and a high collision rate, the DJB2 algorithm distributes data significantly more uniformly and drastically mitigates the occurrence of collisions due to its geometric and multiplicative behavior. It is concluded that minor modifications in mathematical modeling directly impact bit space utilization and computational efficiency, highlighting the logical rigor required in teaching and designing data structures.

Keywords: Hashing algorithms, comparative analysis, hash function, collision rate.

1. INTRODUÇÃO E DEFINIÇÃO DO PROBLEMA

No desenvolvimento de sistemas computacionais, funções de dispersão (hashing) são amplamente utilizadas para transformar informações textuais ou numéricas em valores compactos de tamanho fixo. Esse tipo de processamento está presente em diversas áreas da computação, incluindo:

- ♦ estruturas de dados;
- ♦ mecanismos de busca;
- ♦ indexação de registros;
- ♦ autenticação de usuários;
- ♦ armazenamento de senhas;
- ♦ assinaturas digitais;
- ♦ verificação de integridade de arquivos;
- ♦ sistemas criptográficos.

Uma função hash recebe uma entrada de tamanho variável, normalmente uma cadeia de caracteres (*string*), e produz um valor numérico associado àquela informação. O objetivo principal consiste em distribuir os dados de forma eficiente dentro do espaço numérico disponível, reduzindo a ocorrência de colisões.

O fenômeno denominado colisão ocorre quando duas entradas diferentes produzem o mesmo resultado hash. Embora colisões sejam inevitáveis em qualquer sistema finito de representação numérica, funções de dispersão bem projetadas procuram minimizar sua incidência e distribuir os resultados de maneira uniforme.

Nesta apresentação são analisadas matematicamente duas abordagens distintas de implementação:

- ♦ uma versão baseada em ponderação linear;
- ♦ uma versão baseada em multiplicação sucessiva utilizando o algoritmo DJB2.

A análise compara o comportamento estrutural das duas implementações, observando aspectos relacionados à distribuição numérica, ocupação do espaço de bits e incidência de colisões.

2. ANÁLISE DETALHADA: VERSÃO 1 (PONDERAÇÃO LINEAR)

```
static std::string myHash(std::string mensagem) {
    unsigned long hashCalculado = 0;
    unsigned int peso = 1;

    for (char c : mensagem) {
        hashCalculado += static_cast<unsigned long>(c) * peso;
        peso++;
    }

    std::stringstream hash;
    hash << std::setfill('0') << std::setw(10) << hashCalculado;
    return hash.str();
}
```

A primeira implementação utiliza uma estratégia baseada em soma ponderada de caracteres. Cada símbolo da cadeia de entrada é convertido para seu valor ASCII correspondente e multiplicado por um peso crescente de acordo com sua posição dentro da sequência.

O resultado dessas multiplicações é acumulado em uma variável numérica, chamada *hashCalculado*, responsável pela composição final do hash.

Para uma cadeia de caracteres contendo os caracteres:

"ABC"

o processamento ocorre da seguinte forma:

Caractere	ASCII	Peso	Resultado
A	65	1	65
B	66	2	132
C	67	3	201

O valor final corresponde ao somatório dos resultados:

$$65 + 132 + 201 = 398$$

Matematicamente, o comportamento da função pode ser representado pela seguinte expressão Equação 1):

$$\text{Hash}(S) = \sum_{i=0}^{n-1} \text{ASCII}(S_i) \times (i + 1) \quad (1)$$

Onde:

- S representa a cadeia de caracteres (*string*) de entrada.
- n é o comprimento total da *string* (número de caracteres).
- S_i representa o caractere individual localizado no índice i (com base zero).
- $\text{ASCII}(S_i)$ é a conversão do caractere para seu valor correspondente em ASCII.

Como o algoritmo trabalha apenas com somas sucessivas, o crescimento do valor numérico ocorre de maneira linear. Consequentemente, diferentes combinações de caracteres podem produzir resultados iguais ou muito próximos entre si.

Esse comportamento aumenta significativamente a ocorrência de colisões.

Considere, por exemplo, as seguintes entradas:

"ABCD"
"ACBD"

Embora as cadeias sejam diferentes, pequenas alterações posicionais produzem apenas mudanças proporcionais no somatório final. Como o algoritmo depende exclusivamente de operações aditivas, a capacidade de dispersão permanece limitada.

Outro aspecto importante está relacionado à ocupação do espaço numérico disponível. Apesar do tipo *unsigned long* suportar valores elevados, o crescimento linear restringe a amplitude dos resultados produzidos pelo algoritmo.

A instrução **hash << std::setfill('0') << std::setw(10) << hashCalculado;** não aumenta a dispersão matemática do hash. Ela apenas formata visualmente a saída, preenchendo posições vazias com zeros à esquerda.

Assim, um valor como **398** passa a ser exibido como **0000000398** sem qualquer altera

3. ANÁLISE DETALHADA: VERSÃO 2 (MULTIPLICATIVA SUCESSIVA)

```
static std::string myHash(std::string mensagem) {
    unsigned long hashCalculado = 5381;

    for (char c : mensagem) {
        hashCalculado = ((hashCalculado * 33) + hashCalculado) + static_cast<unsigned long>(c);
    }

    std::stringstream hash;
    hash << std::setfill('0') << std::setw(10) << hashCalculado;
    return hash.str();
}
```

A segunda implementação utiliza uma abordagem multiplicativa inspirada no algoritmo DJB2, desenvolvido na década de 1990 pelo cientista da computação Dan Bernstein.

O algoritmo tornou-se amplamente conhecido em aplicações de dispersão devido à sua simplicidade estrutural, baixo custo computacional e boa distribuição numérica para cadeias de caracteres curtas e médias.

Diferentemente da versão linear anterior, o valor acumulado do hash não cresce apenas por somas sucessivas. A cada iteração, o resultado anterior participa diretamente do próximo cálculo por meio de multiplicações sucessivas, ampliando significativamente a dispersão dos resultados produzidos.

O comportamento matemático da função pode ser representado pela relação (Equação 2):

$$H_i = (H_{i-1} \times 33) + ASCII(S_i) \quad (2)$$

Onde:

- H_i representa o valor atual do hash.
- H_{i-1} representa o valor acumulado o valor acumulado da etapa anterior;
- $ASCII(S_i)$ corresponde ao caractere processado.

Nessa abordagem, cada novo caractere modifica não apenas o valor atual, mas também influencia todas as operações subsequentes do cálculo.

Considere as entradas:

"ABCD"
"DCBA"

Na implementação linear, a alteração de posições produz diferenças relativamente pequenas. Já na implementação multiplicativa, a mudança da ordem dos caracteres modifica completamente a sequência de multiplicações realizadas durante o processamento.

Esse comportamento aumenta significativamente a dispersão dos resultados e reduz a incidência de colisões.

Outro aspecto relevante está relacionado à ocupação do espaço numérico. Como o algoritmo realiza multiplicações sucessivas, o valor acumulado cresce rapidamente, utilizando uma faixa muito maior dos bits disponíveis no tipo *unsigned long*.

Consequentemente, os resultados passam a apresentar distribuição numérica mais ampla e menos previsível.

A constante inicial 5381 e o multiplicador 33 foram definidos empiricamente durante o desenvolvimento do algoritmo DJB2 e apresentam bom comportamento para dispersão de cadeias de caracteres em aplicações gerais de computação.

Embora o DJB2 apresente desempenho significativamente superior ao modelo linear para dispersão em memória e estruturas de dados, ele não deve ser utilizado como algoritmo criptográfico para armazenamento seguro de senhas.

Sistemas modernos de autenticação utilizam algoritmos específicos para proteção de credenciais, como:

- ♦ bcrypt;
- ♦ Argon2;
- ♦ PBKDF2.

O DJB2 pertence principalmente ao contexto de dispersão computacional e indexação eficiente de dados.

4. TABELA COMPARATIVA

A fim de consolidar as diferenças estruturais e comportamentais entre as duas abordagens de dispersão analisadas, estabeleceu-se um conjunto de critérios de corte que contrastam desde a fundamentação aritmética até o impacto prático na ocorrência de colisões. A Tabela 1 sintetiza o mapeamento comparativo entre a implementação por Ponderação Linear (Versão 1) e a implementação Multiplicativa Sucessiva baseada no algoritmo DJB2 (Versão 2).

Tabela 1 – Confronto estrutural e operacional dos algoritmos de hashing analisados.

Critério de Análise	Versão 1: Ponderação Linear	Versão 2: Multiplicativa (DJB2)
Estrutura Matemática	Soma ponderada	Multiplicação sucessiva
Comportamento Matemático	Progressão Aritmética	Progressão Geométrica
Crescimento Numérico	Linear	Multiplicativo
Sensibilidade à Ordem	Baixa	Alta
Ocupação de Bits	Limitada	Ampla
Incidência de Colisões	Elevada	Reduzida
Distribuição Numérica	Restrita	Mais uniforme
Fórmula de Iteração	$H = H + (\text{ASCII} \times \text{peso})$	$H = (H \times 33) + \text{ASCII}$
Valor Inicial (Semente)	0 (Vulnerável a cadeias vazias)	5381 (Número Primo de Dispersão)
Resistência a Anagramas	Nula "AZ" e "ZA" geram colisões	Excelente A inversão altera o produto
Aplicação Principal	Demonstração estrutural	Dispersão prática

A partir dos dados sintetizados na Tabela 1, observa-se que a divergência no desempenho das funções não decorre da complexidade do código em si, dado que ambas operam em complexidade de tempo linear $O(n)$ em relação ao tamanho da cadeia de entrada, mas fundamentalmente da natureza de suas operações aritméticas.

A inclusão de uma operação multiplicativa associada a uma semente de dispersão prima (5381) quebra a previsibilidade aditiva da Versão 1. Enquanto a ponderação linear falha severamente no tratamento de anagramas devido à propriedade comutativa mascarada pelo crescimento estritamente aritmético, a iteração geométrica da Versão 2 garante o efeito avalanche (*avalanche effect*), onde a alteração de um único bit ou da posição de um caractere reorganiza completamente a composição dos bits significativos no acumulador de saída.

5. ANÁLISE EMPÍRICA DE COLISÕES

A avaliação prática e a validação estatística das duas implementações foram realizadas por meio de testes automatizados de força bruta utilizando um universo amostral exaustivo. Nos programas de teste apresentados em apêndice, o algoritmo gera de forma combinatória todas as variações possíveis de cadeias de caracteres com comprimento

fixo de quatro posições ($n = 4$), restritas ao escopo das 26 letras maiúsculas do alfabeto latino.

Esse tipo de análise permite observar experimentalmente:

- ♦ a distribuição dos resultados;
- ♦ a quantidade de colisões;
- ♦ o comportamento matemático da dispersão;
- ♦ a eficiência estrutural de cada abordagem.

Na implementação linear, a taxa de colisões cresce rapidamente devido à limitação do espaço efetivamente ocupado pelos resultados.

Na implementação DJB2, a multiplicação sucessiva amplia significativamente a dispersão numérica, reduzindo a incidência de colisões dentro do mesmo conjunto de testes.

Matematicamente, o tamanho do conjunto de dados injetado nas funções hash é determinado por um arranjo com repetição, totalizando exatamente $26^4 = 456.976$ cadeias (*strings*) testadas.

Esse volume massivo de dados (quase meio milhão de registros) garante o peso estatístico do experimento, permitindo observar com alta precisão o comportamento de dispersão de cada algoritmo sob estresse. Cada uma das 456.976 cadeias geradas foi processada e mapeada em uma estrutura de tabela de dispersão em memória, associando cada valor *hash* calculado ao vetor de *strings* que o originou, viabilizando a contagem exata das colisões ocorridas com o recurso (`std::unordered_map`).

6. CONCLUSÃO

As duas implementações analisadas neste estudo utilizam princípios aritméticos distintos para transformar cadeias de caracteres em valores numéricos identificadores. A primeira versão, fundamentada em uma estratégia de ponderação linear por somas sucessivas, exhibe limitações estruturais severas. Conforme evidenciado na modelagem e nos testes exaustivos, seu crescimento puramente aritmético restringe a amplitude de dispersão e anula a resistência a anagramas, resultando em uma rápida saturação do espaço de busca e em uma elevada taxa de colisões.

Em contrapartida, a versão baseada no algoritmo clássico *DJB2* emprega multiplicações sucessivas sobre o valor acumulado do *hash*, o que otimiza significativamente a distribuição dos dados. Ao introduzir uma semente prima e um comportamento geométrico, o algoritmo alcança o efeito avalanche, ampliando a ocupação do espaço de bits disponível no tipo *unsigned long* e mitigando drasticamente a incidência de colisões.

O confronto entre os dois modelos, validado empiricamente pelo processamento exaustivo de 456.976 vetores de teste em ambiente C++, demonstra como modificações sutis na modelagem matemática de um algoritmo produzem divergências drásticas em sua eficiência computacional, dispersão, ocupação de bits e uniformidade de distribuição

numérica. Em última análise, este trabalho cumpre seu papel didático e científico ao evidenciar a relação de dependência direta entre o rigor lógico da abstração matemática e o comportamento prático de estruturas de dados essenciais à computação contemporânea.

APÊNDICE

São apresentados programas de testes das duas versões de programas com uso de algoritmos de dispersão.

Primeira versão (ponderação linear – colisão 1)

```
// =====
// PROGRAMA      : colisao1.cpp
// DESCRIÇÃO     : Script de análise empírica e quebra por força bruta.
//                Mapeia o comportamento da função de hash de ponderação linear,
//                utilizando tabelas de dispersão em memória (unordered_map)
//                para quantificar a eficiência matemática e a taxa de colisões.
// AUTOR        : Prof. Me. José Augusto Navarro Garcia Manzano
// VERSÃO       : 1.0
// DATA        : Maio / 2026
// DIRETRIZES   : ISO/IEC - Rigor lógico e estrutural para ensino de programação.
// =====

#include <iostream>
#include <string>
#include <sstream>
#include <iomanip>
#include <unordered_map>
#include <vector>

using namespace std;

// A função com ponderação linear
string myHash(string mensagem) {
    unsigned long hashCalculado = 0, peso = 1;

    for (char c : mensagem) {
        hashCalculado += static_cast<unsigned long>(c) * peso;
        peso++;
    }

    stringstream hash;
    hash << std::setfill('0') << std::setw(10) << hashCalculado;
    return hash.str();
}

int main(void) {
    // Mapa que associa o Hash gerado com as strings que o produziram
    unordered_map<string, vector<string>> mapaDeHashes;

    // Gerador simples de strings para teste (combinações de 3 caracteres imprimíveis)
    // Escopo de caracteres ASCII visíveis: do espaço (32) ao 'z' (122)

    unsigned long totalStringsTestadas = 0, totalColisoes = 0;
    double taxaColisao;

    for (char c1 = 'A'; c1 <= 'Z'; c1++) {
        for (char c2 = 'A'; c2 <= 'Z'; c2++) {
            for (char c3 = 'A'; c3 <= 'Z'; c3++) {
                for (char c4 = 'A'; c4 <= 'Z'; c4++) {
                    string texto = "";
                    texto += c1;
                    texto += c2;
                    texto += c3;
                    texto += c4;

                    totalStringsTestadas++;
                    string h = myHash(texto);

                    // Guarda a string no vetor correspondente àquele hash
                    mapaDeHashes[h].push_back(texto);
                }
            }
        }
    }

    cout << "===== " << endl;
    cout << "RESULTADO DO TESTE DE COLISAO - VERSAO 1.0" << endl;
```

```

cout << "=====" << endl;
cout << "Total de strings testadas (tamanho 4): " << totalStringsTestadas << endl;

// Varre o mapa procurando vetores com tamanho maior que 1 (indica colisão)
for (auto const& [hash, strings] : mapaDeHashes) {
    if (strings.size() > 1) {
        totalColisoas += (strings.size() - 1);

        // Exibe as primeiras 5 colisões encontradas para não inundar a tela
        if (totalColisoas <= 5) {
            cout << "Hash: [" << hash << "] colidiu para as entradas: ";
            for (const string& s : strings) {
                cout << "\"" << s << "\" ";
            }
            cout << endl;
        }
    }
}

cout << "-----" << endl;
cout << "Total de colisoas detectadas: " << totalColisoas << endl;
taxaColisao = (static_cast<double>(totalColisoas) / totalStringsTestadas) * 100.0;
cout << "Taxa de colisao no escopo testado: " << taxaColisao << "%" << endl;
cout << "=====" << endl;

cout << endl << "Tecle <Enter> para encerrar... ";
cin.get();

return 0;
}

```

Segunda versão (multiplicativa de constante prima – colisão 2)

```

// =====
// PROGRAMA    : colisao2.cpp
// DESCRIÇÃO   : Script de análise empírica e validação estatística.
//              Mapeia o comportamento da função de hash multiplicativa de
//              constante sucessiva (DJB2), demonstrando visualmente a eficácia
//              da dispersão não linear e a redução drástica de colisões.
// AUTOR       : Prof. Me. José Augusto Navarro Garcia Manzano
// VERSÃO      : 2.0
// DATA       : Maio / 2026
// DIRETRIZES  : ISO/IEC - Rigor lógico e estrutural para ensino de programação.
// =====

#include <iostream>
#include <string>
#include <sstream>
#include <iomanip>
#include <unordered_map>
#include <vector>

using namespace std;

// A função com multiplicativa de constante prima
static std::string myHash(std::string mensagem) {
    unsigned long hashCalculado = 5381;

    for (char c : mensagem)
        hashCalculado = ((hashCalculado * 33) + hashCalculado) + static_cast<unsigned long>(c);

    std::stringstream hash;
    hash << std::setfill('0') << std::setw(10) << hashCalculado;
    return hash.str();
}

int main(void) {
    // Mapa que associa o Hash gerado com as strings que o produziram
    unordered_map<string, vector<string>> mapaDeHashes;

    // Gerador simples de strings para teste (combinações de 3 caracteres imprimíveis)
    // Escopo de caracteres ASCII visíveis: do espaço (32) ao 'z' (122)

    unsigned long totalStringsTestadas = 0, totalColisoas = 0;
    double taxaColisao;

```

```

for (char c1 = 'A'; c1 <= 'Z'; c1++) {
    for (char c2 = 'A'; c2 <= 'Z'; c2++) {
        for (char c3 = 'A'; c3 <= 'Z'; c3++) {
            for (char c4 = 'A'; c4 <= 'Z'; c4++) {
                string texto = "";
                texto += c1;
                texto += c2;
                texto += c3;
                texto += c4;

                totalStringsTestadas++;
                string h = myHash(texto);

                // Guarda a string no vetor correspondente àquele hash
                mapaDeHashes[h].push_back(texto);
            }
        }
    }
}

```

```

cout << "=====" << endl;
cout << "RESULTADO DO TESTE DE COLISAO - VERSAO 2.0" << endl;
cout << "=====" << endl;
cout << "Total de strings testadas (tamanho 4): " << totalStringsTestadas << endl;

// Varre o mapa procurando vetores com tamanho maior que 1 (indica colisao)
for (auto const& [hash, strings] : mapaDeHashes) {
    if (strings.size() > 1) {
        totalColisoes += (strings.size() - 1);

        // Exibe as primeiras 5 colisoes encontradas para não inundar a tela
        if (totalColisoes <= 5) {
            cout << "Hash: [" << hash << "] colidiu para as entradas: ";
            for (const string& s : strings) {
                cout << "\"" << s << "\" ";
            }
            cout << endl;
        }
    }
}

cout << "-----" << endl;
cout << "Total de colisoes detectadas: " << totalColisoes << endl;
taxaColisao = (static_cast<double>(totalColisoes) / totalStringsTestadas) * 100.0;
cout << "Taxa de colisao no escopo testado: " << taxaColisao << "%" << endl;
cout << "=====" << endl;

cout << endl << "Tecle <Enter> para encerrar... ";
cin.get();

return 0;
}

```